

Strukturalni paterni

Adapter patern služi da se postojeći objekat prilagodi za korištenje na neki drugi način, bez mijenjanja same definicije objekta. Na taj način obezbjeđuje se da će se objekti i dalje moći upotrebljavati na način kako su se dosad upotrebljavali, a u isto vrijeme će se omogućiti njihovo prilagođavanje novim uslovima.

Adapter pattern se koristi u sistemu za slanje obavijesti korisnicima. Različiti servisi za slanje obavijesti (npr. email i SMS) imaju različite interfejsa, što otežava njihovu direktnu integraciju u sistem.

Kako bi se riješio ovaj problem, uveden je interfejs **NotificationService** koji definiše jedinstvenu metodu za slanje obavijesti. Konkretno klase **EmailAdapter** i **SMSAdapter** implementiraju ovaj interfejs i prilagođavaju pozive prema specifičnim servisima.

Na ovaj način klasa Obavijest komunicira samo sa interfejsom čime se povećava fleksibilnost i omogućava lako dodavanje novih tipova obavijesti.

Facade patern služi kako bi se klijentima pojednostavilo korištenje kompleksnih sistema. Klijenti vide samo fasadu, odnosno krajnji izgled objekta, dok je njegova unutrašnja struktura skrivena. Na ovaj način smanjuje se mogućnost pojavljivanja grešaka jer klijenti ne moraju dobro poznavati sistem kako bi ga mogli koristiti.

Procesi rezervacije i kupovine uključuju više klasa, što povećava složenost sistema. Uvođenjem klasa **RezervacijaFacade** i **KupovinaFacade** omogućava se da klasa Klijent komunicira sa sistemom preko jednostavnog interfejsa. Facade klase interno koriste više klasa kako bi izvršile potrebne operacije. Na ovaj način smanjuje se kompleksnost i povećava preglednost sistema.

Pored implementiranih Adapter i Facade paterni, u sistemu bi se mogla razmotriti i primjena drugih strukturalnih paterni radi dodatnog unapređenja fleksibilnosti sistema.

Decorator patern služi za omogućavanje različitih nadogradnji objektima koji svi u osnovi predstavljaju jednu vrstu objekta (odnosno, koji imaju istu osnovu). Umjesto da se definiše veliki broj izvedenih klasa, dovoljno je omogućiti različito dekoriranje objekata (tj. dodavanje različitih detalja), te se na taj način pojednostavljuje i rukovanje objektima klijentima, i samo implementiranje modela objekata. Decorator patern bi se mogao primijeniti u okviru klase *Usluga*, gdje bi omogućio dodavanje dodatnih funkcionalnosti osnovnoj usluzi, kao što su npr. popusti. Na ovaj način bi se izbjeglo kreiranje velikog broja kombinovanih klasa usluga, a omogućila bi se fleksibilna nadogradnja postojećih funkcionalnosti.

Bridge patern služi kako bi se apstrakcija nekog objekta odvojila od njegove implementacije. Ovaj patern je veoma važan jer omogućava ispunjavanje Open-Closed SOLID principa, odnosno uz poštivanje ovog paterni omogućava se nadogradnja klasnog modela u budućnosti te osigurava da se neće morati vršiti određene promjene u postojećim klasama.

Bridge patern bi se mogao primijeniti u dijelu sistema koji se odnosi na usluge, gdje bi se apstrakcija usluge odvojila od njenog konkretnog tipa ili varijante. Na primjer, usluga šišanja bi mogla biti razdvojena po tipu, kao što su šišanje kratke kose, šišanje duge kose ili šišanje uz stilizovanje kose. Na taj način se omogućava nezavisno proširivanje različitih tipova usluga i njihovih varijanti, bez potrebe za kreiranjem velikog broja kombinovanih klasa, čime se sistem čini fleksibilnijim i lakšim za održavanje.

Proxy patern služi za dodatno osiguravanje objekata od pogrešne ili zlonamjerne upotrebe. Primjenom ovog paterni omogućava se kontrola pristupa objektima, te se onemogućava manipulacija objektima ukoliko neki uslov nije ispunjen, odnosno ukoliko korisnik nema prava pristupa traženom objektu.

U okviru sistema, Proxy patern bi se mogao primijeniti u dijelu koji se odnosi na narudžbe ili zakazivanje usluga, gdje bi proxy klasa kontrolisala pristup servisima za kreiranje narudžbi i provjeru termina. Na primjer, prije kreiranja narudžbe može se provjeriti da li je korisnik autentifikovan i da li su ispunjeni svi uslovi za izvršenje operacije, nakon čega se zahtjev proslijeđuje stvarnom servisu.

Composite patern služi za kreiranje hijerarhije objekata. Koristi se kada svi objekti imaju različite implementacije nekih metoda, no potrebno im je svima pristupati na isti način, te se na taj način pojednostavljuje njihova implementacija.

Composite patern bi se mogao koristiti u dijelu sistema koji se odnosi na proizvode i korpu. Ovaj patern bi omogućio da se pojedinačni proizvodi i grupe proizvoda (paketi) tretiraju na isti način. Time bi se omogućilo jednostavno upravljanje složenim strukturama proizvoda unutar korpe i narudžbi.